Lecture 5 · Part I · Lecture notes

Regularisation and the bias–variance trade-off

Dataset: Titanic, 891 passengers

How to use these notes

The primary text for this lecture is Géron, Chapter 4. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 Two questions the sweep cannot answer

Lecture 4 fitted a logistic regression to 712 passengers and then added polynomial capacity, recording both curves at every degree.

Degree	Columns	Training log loss	Held-out log loss
1	22	0.395	0.468
2	32	0.381	0.467
3	52	0.355	0.538
5	143	0.286	1.922

One curve falls forever; the other turns and climbs. Two questions follow, and they are different questions. *The held-out error is 0.468 and the training error is 0.395 — where does the difference come from?* And *0.468 against an anchor of 0.666 — how far down could it ever go?*

The two have different repairs, and the repairs point in opposite directions. Attack the wrong one and you spend an afternoon making the model worse.

2 Derivation: the bias–variance decomposition

2.1 First, what is random

Not the passenger. Fix one passenger x and ask about them repeatedly. What varies is the *training set* D — you happened to draw these 712 rows — and the *label* y, because two people with identical recorded values did not both survive. Every expectation below is over those two sources and nothing else; the model is deterministic given D.

Symbol	Meaning	Observed?
$\hat{p}_D(x)$	what your model predicts, having seen D	yes
$\bar{p}(x) = \mathbb{E}_D[\hat{p}_D(x)]$	the average prediction over all training sets	no — estimated
$p^*(x)$	the true probability that this passenger survives	never

$y \in \{0, 1\}$ is what actually happened. It is never p^*, and that gap becomes the third term.

2.2 The decomposition

Add and subtract, twice

Hold y fixed and average over D only:

$$(y - \hat{p}_D)^2 = \Big(\underbrace{y - \bar{p}}_{\text{does not depend on } D} + \underbrace{\bar{p} - \hat{p}_D}_{\text{mean 0}} \Big)^2 .$$

The cross term is $2(y - \bar{p})\,\mathbb{E}_D[\bar{p} - \hat{p}_D] = 0$, because $\bar{p}$ is *defined* as $\mathbb{E}_D[\hat{p}_D]$. So

$$\mathbb{E}_D\big[(y - \hat{p}_D)^2\big] = (y - \bar{p})^2 + \underbrace{\mathbb{E}_D\big[(\hat{p}_D - \bar{p})^2\big]}_{\textbf{variance}} .$$

Repeat the trick on $(y - \bar{p})^2$, inserting p^*. Since $\mathbb{E}_y[y] = p^*$ the cross term vanishes again, and $\mathrm{Var}(y) = p^*(1 - p^*)$ because y is Bernoulli:

$$\mathbb{E}_y\big[(y - \bar{p})^2\big] = \underbrace{(p^* - \bar{p})^2}_{\textbf{squared bias}} + \underbrace{p^*(1 - p^*)}_{\textbf{noise}} .$$

$$\mathbb{E}\big[(y - \hat{p}_D)^2\big] = (p^* - \bar{p})^2 + \mathbb{E}_D\big[(\hat{p}_D - \bar{p})^2\big] + p^*(1 - p^*).$$

Term	Means	Cured by
squared bias	wrong on average — the model family cannot reach the truth	more capacity
variance	unstable — a different sample gives a different answer	less capacity, or more data
noise	the label is not a function of the recorded columns	nothing

Generally the identity is $\mathbb{E}[(y - \hat{f})^2] = \text{bias}^2 + \text{variance} + \mathrm{Var}(y \mid x)$; $p^*(1 - p^*)$ is that last term for a Bernoulli label, one instance of the identity rather than the identity itself.

The first two repairs point in opposite directions. That is the whole difficulty, and it is why it is called a trade-off.

An honest caveat before we measure anything

The identity is exact *for squared error*. Our headline metric is log loss, and log loss does not split into three additive terms like this.
So the decomposition is measured on the *Brier score* — the squared error of a probability, which Lecture 4 already reported alongside log loss. The shape of the answer transfers; the numbers are Brier numbers and are labelled as such. Saying "bias–variance" about a log-loss curve is common and slightly wrong. Doing it knowingly, and saying which metric you decomposed, is the difference.

3 Measuring all three terms

3.1 The term you cannot fix

p^* is never observed — but where several passengers share exactly the same recorded values, every model of those columns *must* give them the same number. Whatever their outcomes do inside that cell is a floor.

Grouping on sex, class, age band, family band and port gives 133 distinct cells, of which 102 hold at least two passengers, covering 858 people. Using $k(m-k)/(m(m-1))$, the unbiased estimator of $p(1-p)$ from m Bernoulli draws, the floor is **0.121** Brier — measured, not assumed.

56 of the 102 shared cells are mixed: 657 passengers, 74% of everyone on board, sit in a cell whose answer cannot be certain. The starkest is 62 third-class men aged 26 to 40 travelling alone from Southampton, with every recorded value identical; 13 of them survived, and no model of these columns can separate them.

It is an *under*-estimate: two passengers in the same cell may still differ in ways the manifest never recorded. The real floor is at least this high.

3.2 The other two, by resampling

Draw 200 training sets of 400 rows each from the 712 we hold, fit the pipeline on each, and predict the same 179 test passengers. That is the expectation over D, done by brute force — the only way to see a quantity defined over training sets you did not draw. Bias and noise cannot be separated without p^*, so they are reported as one column, and the floor above is what splits them.

Check the identity before believing the plot

```
>>> abs(total.mean() - variance.mean() - bias2_pn.mean())
2.7755575615628914e-17
```

Machine epsilon on a double. The three columns below are not a *model* of the error — they are the error, rearranged.
Do this every time you implement a decomposition. If the residual is not at floating-point noise you have a bug, and the picture will still look plausible.

Degree	bias^2 + noise	variance	total
1	0.130	0.006	0.136
2	0.134	0.011	0.146
3	0.138	0.029	0.167
4	0.163	0.059	0.222
5	0.181	0.068	0.248
6	0.182	0.071	0.253

Read the first column: 0.130 at degree 1, against a measured noise floor of 0.121. Almost all of it is noise, so the squared bias of a degree-1 logistic model on these features is under 0.01.

There was never much bias to buy back

Every extra degree bought variance and nothing else — the variance term is multiplied by twelve between degree 1 and degree 6 while everything else is nearly flat. That one sentence decides the rest of the lecture.

3.3 Putting it back on the two curves

The training curve falls forever because it is measured on the rows that were fitted, so it sees no variance term at all. The held-out curve is the sum of all three terms, and from degree 3 the variance term dominates.

The identification, stated once

The vertical gap between the two curves *is* the variance term. At degree 1 the gap is 0.073; at degree 5 it is 1.636. The gap grew by a factor of 22 while nothing about the data changed.

There are only two shapes, and telling them apart takes ten seconds. Both curves plateau high and together: *bias* — the model is too rigid, and more data changes nothing. A gap still open at the last point: *variance* — more data would help, and you may not be able to get any. The repairs are opposites, and applying the left-hand repair to a right-hand curve is the most common wasted afternoon in applied machine learning.

3.4 Learning curves, and a knob we do not have

A learning curve puts *rows* on the x-axis and answers a question the degree sweep cannot: would more data help?

	Degree 1	Degree 5
Held-out log loss at 76 rows	2.853	9.889
Training log loss at 640 rows	0.395	0.286
Held-out log loss at 640 rows	0.474	1.911
Gap at 640 rows	0.078	1.625

The degree-5 held-out curve falls from 9.9 to 1.9 as rows arrive and is still falling steeply at 640. Extrapolating, that model would need several times our dataset before becoming competitive.

There are 891 passengers on the Titanic

There will never be more. Rows are not a knob we can turn, so the only repair left is to shrink the model.

A *validation* curve puts one hyperparameter on the x-axis instead, and answers which setting is right and whether the optimum is interior. Read the ends first: a minimum sitting on the edge of the range means the range was too narrow, and the answer you read off it is an artefact of where you stopped looking.

4 Three faults at degree 5

Degree 5 scored 1.922 — almost three times worse than *saying nothing at all*. Not worse than the best model; worse than the anchor of 0.666, worse than a model that has learned only how many people died.

How is that possible when held-out accuracy was 76.0%? Because $-\log(0.01) \approx 4.6$: a single passenger given probability 0.01 who then survives contributes 4.6 to the average on its own. Accuracy records that passenger as one error and moves on. Log loss is unbounded above — not a defect, but the metric saying that confident-and-wrong is a different failure from wrong.

4.1 Fault 1 — variance

	Degree 1	Degree 5
Columns	22	143
Training rows	712	712
Rows per weight	32	5
Variance term, Brier	0.006	0.068

Five rows per weight. Move one passenger between folds and the fitted surface moves with them. This is the diagnosis the derivation was built to deliver — and two more faults sit underneath it.

4.2 Fault 2 — the minimiser does not exist

If some $\boldsymbol{\theta}$ separates the classes perfectly in the expanded space, then $\sigma(c\,\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x})$ tends to the right label for every row as $c \to \infty$, so the training log loss falls monotonically towards zero as $\|\boldsymbol{\theta}\| \to \infty$. The infimum is never attained.

Degree	1	2	3	4	5	6
Converged	yes	yes	yes	no	no	no
Largest $\lvert\theta_j\rvert$	4.87	5.43	6.32	18.70	11.07	3.98

4.3 Fault 3 — the minimiser is not unique

`FamilySize = SibSp + Parch + 1` exactly, on every row. Standardise the five numeric columns, form $\mathbf{X}^\top\mathbf{X}$ and look at its spectrum:

```
>>> np.linalg.eigvalsh(Z.T @ Z)
array([1.81e-12, 4.29e+02, 5.43e+02, 7.90e+02, 1.80e+03])
```

The smallest eigenvalue is zero to the precision the arithmetic allows: four columns of information in five columns of data, condition number about 9×10^{14}. A double carries about 10^{16} of relative precision, so roughly one significant digit survives the solve.

5 One term repairs all three

5.1 What adding $\alpha\mathbf{I}$ does to the spectrum

Three lines, and Lecture 2 is closed

If $\mathbf{X}^\top\mathbf{X}\mathbf{v} = \lambda\mathbf{v}$ then

$$\big(\mathbf{X}^\top\mathbf{X} + \alpha\mathbf{I}\big)\mathbf{v} = (\lambda + \alpha)\mathbf{v}\,:$$

same eigenvectors, every eigenvalue moved up by exactly α. $\mathbf{X}^\top\mathbf{X}$ is positive semi-definite, so $\lambda \geq 0$, so $\lambda + \alpha \geq \alpha > 0$. No eigenvalue is zero, so the matrix is invertible *for every* $\alpha > 0$.

Lecture 2 derived $\hat{\boldsymbol{\theta}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top y$ and left open what to do when that inverse does not exist. Those three lines are the answer.

It also bounds the conditioning:

$$\kappa\big(\mathbf{X}^\top\mathbf{X} + \alpha\mathbf{I}\big) = \frac{\lambda_{\max} + \alpha}{\lambda_{\min} + \alpha} \leq \frac{\lambda_{\max} + \alpha}{\alpha},$$

and the bound on the right does not mention $\lambda_{\min}$ at all. Whatever the data do — exact collinearity included — the conditioning is controlled by a number you choose. At $\alpha = 1$ the condition number of our singular matrix falls from 9×10^{14} to 1798. The dependence is not repaired; it is *dominated*. And since κ set the step count for gradient descent in Lecture 4, the penalty is not only a statistical device — it is what makes the optimisation tractable.

5.2 The idea, and the three penalties

Stop asking for the best fit. Ask for the best fit *among small models*:

$$J(\boldsymbol{\theta}) = \underbrace{L(\boldsymbol{\theta})}_{\text{fit the data}} + \underbrace{\alpha\,\Omega(\boldsymbol{\theta})}_{\text{stay small}}.$$

Setting $\alpha = 0$ recovers Lecture 4 exactly.

Ridge uses $\Omega = \frac{1}{2}\|\mathbf{w}\|_2^2$, where $\mathbf{w}$ is $\boldsymbol{\theta}$ without the bias — the intercept is *not* penalised, because shrinking it would pull every prediction towards zero rather than towards the mean. On the log loss the objective becomes *coercive*: as $\|\mathbf{w}\| \to \infty$ the penalty tends to infinity while the log loss is bounded below by zero, so a minimiser exists, and strict convexity makes it unique. Faults 2 and 3 are both gone, and we have not looked at the data once.

Lasso uses $\sum_j |\theta_j|$ instead, and the word *selection* in its name is the point.

Why ℓ_1 produces exact zeros and ℓ_2 does not

Look at the gradient of the penalty near $\theta_j = 0$. For ridge, $\partial(\theta_j^2)/\partial\theta_j = 2\theta_j \to 0$: the push towards zero fades as you approach it, so you arrive only in the limit. For lasso, $\partial|\theta_j|/\partial\theta_j = \pm 1$: the push keeps its size right up to zero, so if the data pulls with less than α the weight reaches zero and stops.
At $\theta_j = 0$ the absolute value is not differentiable; the subgradient is the whole interval $[-1, 1]$, and any data pull inside that interval is held at exactly zero.

Elastic net mixes the two with a ratio r, and is preferable to lasso when the features are correlated or outnumber the instances, because lasso behaves erratically there. One of those holds here: Age and Age^2 are about as correlated as two columns get.

A penalty requires scaling — not as a nicety

The penalty $\sum \theta_j^2$ treats every weight identically. Rescale a column by c and its fitted weight scales by $1/c$, so its contribution to the penalty scales by $1/c^2$.
Changing the *unit* of a column therefore changes which coefficients ridge decides to shrink, and nothing in the output says so. `StandardScaler` is already inside the pipeline; a penalty is the reason it is not optional.

And a trap in the API: `LogisticRegression` takes $C = 1/\alpha$, not α. Small `C` is strong regularisation. The default is `C=1.0` with `penalty="l2"`, so scikit-learn's logistic regression is regularised unless you say otherwise — Lecture 4 switched it off on purpose, and that choice is what left three faults to repair.

5.3 The sweep, on the model that failed

Penalty at degree 5	Best C	Log loss	Non-zero weights
none — Lecture 4	—	1.922	143
ridge	0.01	0.706	143
elastic net, $r = 0.5$	0.0001	0.684	3
lasso	0.00032	0.683	3

Ridge alone removes 1.215 of log loss. Lasso removes 1.239 and throws away 140 of the 143 columns while doing it. Both minima sit near the strong-penalty end of the range, so widen the grid before believing either — and the lasso curve is nearly flat there, so "the best `C`" is a plateau, not a point.

5.4 And now the uncomfortable part

Model	Cross-validated log loss
Degree 5, lasso, tuned	0.683
Degree 2, no penalty at all	0.467

The best regularised degree-5 model does not beat the plain degree-2 model we already had. Regularisation repaired an over-complex model; it did not turn it into a better one than the simple model — because there was never much bias to buy back. The decomposition said so twenty minutes earlier, and this is what that sentence looks like in a table.

So what is a penalty actually for?

It makes the problem *well-posed*: a unique minimiser exists for every $\alpha > 0$, whatever the columns do. It makes capacity a *continuous* dial, where degree is an integer. It is *insurance* against not knowing the right capacity in advance, which is the usual case. And it *selects* — lasso's three surviving columns are a hypothesis about which features matter, generated by the fit.

What it is not is a way to make an over-parameterised model beat a well-chosen one on a dataset this small.

5.5 Early stopping

Fit iteratively and stop before the optimiser gets where it is going. Descent starts at $\boldsymbol{\theta} = \mathbf{0}$ — the smallest possible model — and each epoch moves the weights outward, so held-out error falls, reaches a minimum, and then rises. No penalty term and no extra term in the objective: just a stopping rule, whose knob is the epoch count.

	Epoch 105	Epoch 500
Training log loss	2.458	2.219
Validation log loss	2.371	3.645

Epoch 105 is the best model this run ever holds, and running to 500 costs 1.274 of log loss without ever announcing it. Read the table honestly, though: these are bad numbers in absolute terms. Plain SGD at a constant learning rate on 143 correlated columns is a poor optimiser, and this shows the *shape*, not a competitive model. The training curve is also noisy, so "stop at the minimum" needs a patience rule in practice. And the stopping epoch is chosen by looking at held-out data, so it must be chosen on validation rows, never on the test set.

Repair	Keeps $\|\boldsymbol{\theta}\|$ small by	Zeros?
Ridge	penalising $\sum \theta_j^2$	no
Lasso	penalising $\sum \|\theta_j\|$	yes
Elastic net	both, mixed by r	yes
Early stopping	starting at zero and not going far	no

All four reduce variance by shrinking the set of models the fit may reach. All four increase bias. Which is why all four need α — or an epoch count — chosen by cross-validation.

6 Choosing α, and the test set once

A hyperparameter chosen on the rows you then report

Looping over 17 values of `C`, scoring each on the test set and keeping the best, produces a minimum over seventeen *noisy* estimates — and the minimum of noisy estimates is biased downward even when every estimate is individually unbiased.
The code never writes to the test set. It reads it seventeen times, and reading is enough.

Measured over 20 splits, against the honest version that picks `C` by 5-fold cross-validation inside the training part:

Reported number	Log loss	Spread over 20 splits
Chosen on the test set, scored on the test set	0.451	0.049
Chosen by cross-validation, scored on the test set	0.470	0.045

Optimism of 0.019 of log loss, about 4.1%. On 13 splits of 20 the first number is the flattering one, and the two procedures picked the same `C` on 7 splits of 20.

Read that result in both directions. It is *small* — 0.019 against a split-to-split spread of 0.045, invisible on one split. And it is *real and one-sided*: it never averages away, and it grows with the number of candidates you try. Seventeen candidates is a small search; search a thousand, which a randomised search does in an afternoon, and the same procedure hands you a number that is confidently wrong. The size of the error scales with how hard you looked.

The procedure without the problem is `GridSearchCV`: same seventeen candidates, same runtime, *two* numbers instead of one. Report both. The second is allowed to be worse than the first, and if it is much worse that is itself the finding.

6.1 The honest numbers

179 passengers, untouched since the split in Lecture 4. Five candidates, all of whose hyperparameters were fixed before this cell ran.

Model	Log loss	Brier	Accuracy
Degree 5, no penalty — where the sweep ended	1.727	0.189	74.9%
Degree 5, ridge, tuned	0.804	0.173	77.1%
Degree 5, lasso, tuned	0.677	0.244	65.9%
Degree 1, scikit-learn defaults	0.433	0.134	82.7%
Degree 2, no penalty — the winner	0.427	0.134	82.7%

The winner improves on the degree-5 model Lecture 4 finished with by 1.300 of log loss. And row four deserves as much attention: `LogisticRegression()` with every default left alone — degree 1, `C=1.0`, `penalty="l2"` — scores 0.433, statistically the same model as the winner.

The result nobody wanted

Ninety minutes of ridge, lasso, elastic net and early stopping, and the best system on the test set is a two-line model anyone could have written before Lecture 4 started.
Report it anyway. The alternative is choosing the interesting model over the better one.

Then what was any of it for? You now know *why* degree 2 wins, and can say it in three terms rather than as a preference. You know the floor is 0.121 Brier and that no model of these columns goes below it. You know the degree-5 model was not merely worse but *ill-posed*, and exactly which term repairs that. And you have four repairs, measured, for the next dataset — where 143 columns will arrive with a hundred times as many rows and the answer will be different.

A negative result you can explain is a result. A positive result you cannot explain is a liability.

6.2 Is 0.427 good?

Cross-validated, the model beats the anchor by 0.199 of log loss and the majority class by 20.5 points of accuracy; both are real. Brier 0.134 against a measured floor of about 0.121 leaves roughly 0.013 of Brier score on the table, in total, for *any* model of these columns. And there are 179 test passengers, so a difference of a point or two of accuracy is three or four people and is not a difference.

That last sentence belongs in the report. A test set of 179 cannot support the comparisons people will want to make with it.

Regularisation checklist

1. Is the data scaled, inside the pipeline, before a penalty is applied?
2. Is the bias term excluded from the penalty?
3. Was α (or `C`) chosen on validation folds, never on the test set?
4. If the search sat at the edge of the grid, was the grid widened?
5. Is the *unregularised* model in the comparison table, so the penalty has to earn its place?
6. Was the stopping epoch, if any, chosen on held-out rows?

The fifth is the one people skip, and it is the one that would have saved an hour today.

7 The notebook

What to change

In the penalty sweep, widen `np.logspace(-4, 4, 17)` to `(-8, 4, 25)`. Does the lasso minimum move? If it does, the number in §6.3 was an artefact of where we stopped looking — and you have just done checklist item four on the course's own result.

8 Exercises

1. Write the bias–variance decomposition of the expected squared error, and name the one term no model can reduce. [4]

2. A learning curve shows training and validation error both high and close together, and flat as data is added. Diagnose it, and say whether more data would help. [5]

3. Ridge adds αI to $X^\top X$ before inverting. Give the numerical consequence, in terms of the condition number. [4]

4. Why must features be scaled before ridge or lasso, when they need not be for ordinary least squares? [4]

5. You tune α over a grid using cross-validation, then report the best cross-validated score as your estimate of future performance. Name the error, and quantify its direction. [5]

Summary

Expected squared error splits exactly into squared bias, variance and irreducible noise — over the training set and the label, never over the test point. The gap between a training curve and a held-out curve *is* the variance term; curves plateauing together mean bias instead. On this data the noise floor is measurable at 0.121 Brier, from passengers whose recorded values are identical, and the squared bias of a degree-1 model is under 0.01 — so every extra degree bought variance and nothing else.

Degree 5 failed three ways at once: high variance at five rows per weight, no minimiser under separation, and no unique minimiser under an exact collinearity whose smallest eigenvalue was 1.81×10^{-12}. Adding $\alpha\mathbf{A}$ repairs the last two for every $\alpha > 0$, which is Lecture 2's open question answered in three lines, and bounds the condition number by a quantity you choose.

Ridge shrinks, lasso selects, elastic net does both, early stopping does it by not arriving. None of them beat choosing the right capacity: the winner on the test set was degree 2 with no penalty at 0.427, against 0.683 for the best tuned degree-5 model. A penalty is a repair, not an upgrade. And a hyperparameter chosen on the test set cost 0.019 of log loss here, one-sided, growing with the size of the search.